

Analysis of Rendering Algorithms Using Asymptotic Notation

Source Code:

```
#include <stdio.h>

#include <math.h>

int main()
{
    int inputs[] = {10, 100, 1000, 10000};
    int i;
    printf("-----\n");
    printf(" Input\t\tO(n)\t\tO(n log n)\t\tO(n^2)\n");
    printf("-----\n");
    for(i = 0; i < 4; i++)
    {
        int n = inputs[i];
        double linear = n;
        double nlogn = n * (log(n) / log(2));
        double quadratic = (double)n * n;
        printf("%5d\t\t%.0f\t\t%.2f\t\t%.0f\n",
            n, linear, nlogn, quadratic);
    }
    printf("\n-----\n");
    printf("\nObservation:\n");
    for(i = 0; i < 4; i++)
    {
        int n = inputs[i];
        double linear = n;
        double nlogn = n * (log(n) / log(2));
```

```
double quadratic = (double)n * n;
if(linear < nlogn && nlogn < quadratic)
{
    printf("For n = %d :  $O(n) < O(n \log n) < O(n^2)$ \n", n);
}
}
printf("\nConclusion:\n");
printf("As the input size increases,  $O(n)$  grows the slowest and is the most efficient.\n");
printf(" $O(n \log n)$  is moderately efficient, while  $O(n^2)$  grows rapidly.\n");
return 0;
}
```